

CSCE 611 OPERATING SYSTEMS · MODULE 02 · LECTURE NOTES

Processes, Threads, and Context Switching

The illusion of having the machine to yourself.

MODULE OUTCOMES

- Describe a process's state and where the kernel keeps it.
- Implement a context switch and explain each saved register.
- Distinguish processes from threads and state what each shares.
- Explain fork, exec, and why they are separate calls.
- Quantify context switch cost, including the indirect costs.

1 What a process consists of

A process is a program in execution together with everything the kernel needs in order to stop it and later resume it as though nothing had happened. That state lives in the **process control block**.

Component	Contents	Why it is needed
Register state	Program counter, stack pointer, general registers, status register.	Execution resumes exactly where it stopped.
Address space	The root of the page table.	Installed on switch; defines what memory the process can see (CSCE 614 Module 07).
Open file table	File descriptors and their offsets.	A read must continue where the last one stopped.
Kernel stack	A separate stack used while in kernel mode.	A system call that blocks must keep its kernel-side call frames. Without a per-process kernel stack, a blocked syscall could not be resumed.
Scheduling state	State, priority, accumulated run time.	Module 03.
Identity	PID, parent PID, user ID, working directory.	Permissions and process relationships.

1.1 States

State	Meaning	Leaves by
RUNNING	Executing on a CPU now.	Blocking (→ SLEEPING), being preempted (→ RUNNABLE), or exiting (→ ZOMBIE).
RUNNABLE	Ready to run; waiting only for a CPU.	Being scheduled (→ RUNNING).
SLEEPING	Waiting for an event: I/O completion, a lock, a child's exit.	The event occurring (→ RUNNABLE).
ZOMBIE	Has exited; the exit status has not yet been collected.	The parent calling wait().

WHY ZOMBIES EXIST

When a process exits, its exit status must remain available until the parent asks for it — the parent may not be running at that moment, and may not ask for some time.

So the process's memory, files, and address space are released immediately, and a small record survives holding the exit status. That record is the zombie.

A zombie is therefore not a leak or a malfunction; it is a deliberate data structure. An *accumulation* of zombies is a bug — a parent that never calls wait(). If the parent dies first, the child is reparented to init, which reaps it.

2 The context switch

```
# swtch(struct context *old, struct context *new)
swtch:
    sd ra, 0(a0)      # save return address into *old
    sd sp, 8(a0)      # save stack pointer
    sd s0, 16(a0)     # save callee-saved registers s0-s11
    ...              # (caller-saved ones are already spilled
    ...              # by the calling convention)

    ld ra, 0(a1)       # load the NEW context's return address
    ld sp, 8(a1)       # load the NEW context's stack pointer
    ld s0, 16(a1)
    ...
    ret               # return using the newly loaded ra
```

CALLED IN ONE PROCESS, RETURNS IN ANOTHER

swtch is entered on process A's stack and leaves on process B's. The ret at the end uses the return address just loaded from B's saved context, so control resumes wherever B last called swtch.

From B's point of view, its own call to swtch — made possibly milliseconds ago — has simply returned.

Only **callee-saved** registers are stored. The calling convention guarantees that caller-saved registers were already spilled to the stack by whoever called swtch, and that stack is saved along with the stack pointer. This is a genuine economy, not a shortcut.

2.1 Why there are two switches, not one

THE SCHEDULER NEEDS ITS OWN STACK

A process cannot switch directly to another process: it would have to execute the switching code on its own kernel stack while that stack is being abandoned.

xv6 therefore switches **process** → **scheduler** → **process**. Each CPU has a dedicated scheduler context with its own stack. The outgoing process calls swtch into the scheduler; the scheduler loop selects the next runnable process and calls swtch into it.

Two switches rather than one, and at every moment the code that is running has a valid stack that nobody is about to reuse. Every real kernel does something equivalent, whatever it calls it.

3 Threads

	Processes	Threads
Address space	Separate.	Shared.
File descriptors	Separate.	Shared.
Communication	IPC: pipes, sockets, shared memory segments.	Write to a variable.
Fault isolation	A crash affects one process.	A crash takes down every thread in the process.
Switch cost	Page table change → TLB flush.	No page table change → no flush.
Default posture	Isolated.	No isolation whatsoever.

The switch-cost row is the quantitative heart of the comparison. Changing address space invalidates TLB entries (CSCE 614 Module 07), so the new process begins with no cached translations and pays page walks until the TLB refills. Modern hardware mitigates this with address-space identifiers that tag TLB entries, avoiding a full flush — but the cache is still cold either way.

3.1 Where threads live

Model	How	Consequence
1:1 (kernel threads)	Each user thread is a kernel-scheduled entity.	True multicore parallelism. Each switch is a kernel operation. Linux and Windows use this.
N:1 (user threads)	A library multiplexes many threads onto one kernel thread.	Switches are a function call — very cheap. One blocking system call blocks every thread , and no multicore parallelism is possible.
M:N	Many user threads over several kernel threads.	Attempted repeatedly (Solaris, early Rust) and generally abandoned: the scheduler interactions are complex and the benefit modest.
Async runtimes	N:1 or M:N, with the runtime owning all I/O and making it non-blocking.	The modern answer. Go goroutines, Rust async, Java virtual threads. Cheap switches <i>and</i> no blocking problem, because the runtime controls the blocking points.

4 fork and exec

Unix creates processes with two calls where most systems have one. `fork()` duplicates the calling process; `exec()` replaces the program running in the current process, keeping the process itself.

```

pid_t pid = fork();
if (pid == 0) {
    // Child. The gap between fork and exec is the useful part:
    close(1);
    open("out.txt", O_WRONLY | O_CREAT, 0666); // becomes fd 1
    setrlimit(...);                          // resource limits
    signal(SIGINT, SIG_DFL);                  // reset handlers
    execv("/bin/ls", argv);                   // replace the program
    _exit(1);                                // exec failed
} else {
    wait(&status);
}

```

WHY TWO CALLS IS THE BETTER DESIGN

A single `spawn(program, args, ...)` call would need a parameter for every adjustment anyone might want: redirect this descriptor, set that limit, change the working directory, adjust the signal mask, join this namespace. The parameter list grows without bound, and anything the designers did not anticipate is impossible.

With `fork/exec`, the child performs those adjustments with **ordinary code**, using the same system calls available everywhere else. A shell implements pipes, redirection, and job control entirely in this gap, with no special API.

The cost is that `fork` is expensive to implement well, and that it interacts badly with threads — only the calling thread survives in the child, so locks held by other threads are left held forever. This is why `posix_spawn` and `vfork` exist, and why `fork-in-a-threaded-program` is treacherous.

4.1 Copy-on-write

THE OPTIMISATION THAT MAKES FORK VIABLE

Duplicating the parent's entire address space would make `fork` cost time proportional to the parent's memory footprint — and in the common `fork-then-exec` pattern, essentially all of that work is discarded microseconds later.

So the kernel does not copy. It maps the same physical pages into both address spaces, marks them **read-only in both**, and increments a per-page reference count.

The first time either process writes, the hardware raises a page fault. The kernel allocates one new page, copies the contents, makes the copy writable in the faulting process, decrements the reference count, and resumes the instruction.

Only pages actually written are ever copied. `fork` becomes proportional to the page table size rather than the memory size, and `fork+exec` copies almost nothing. This is required in Project 2 and is a direct application of the page-fault machinery from CSCE 614 Module 07.

5 What a switch really costs

Cost	Typical magnitude	Cause
Direct	1–2 μ s	Saving and restoring registers, scheduler bookkeeping, the trap in and out.
TLB	Hundreds of cycles, spread over the next millisecond	Address space change invalidates translations; subsequent accesses pay page walks (CSCE 614 Module 07).
Cache	Frequently the largest term	The incoming process's working set is not in L1 or L2. Every access misses until the cache refills, at ~ 250 cycles for a main-memory miss (CSCE 614 Module 05).
Branch predictor	Moderate	Accumulated history now refers to different code (CSCE 614 Module 04).

The indirect costs typically exceed the direct cost by several times, and they are invisible in any measurement that only times the switch itself. This is the practical reason scheduling quanta are measured in milliseconds rather than microseconds: the switch must be amortised over enough useful work to be worth paying for. Module 03 builds on this directly.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
MIT 6.1810 — lectures and lab on scheduling / swtch	The context switch traced through real code, with the two-switch design explained.
xv6 book — Chapter 7 (Scheduling)	swtch, the scheduler loop, and the per-CPU scheduler context.
OSTEP — Chapters 4–6 (Processes, the Process API, Limited Direct Execution)	The clearest written treatment of fork/exec and of why the API is shaped as it is.
Baumann, Appavoo, Krieger, Roscoe — A fork() in the road (free)	A serious argument that fork was a mistake. Read it after you have understood why it was a good idea — the disagreement is instructive.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Read xv6's swtch.S and scheduler(). Set a GDB breakpoint in swtch and single-step across a switch, watching the stack pointer change process.
- 2 Instrument xv6 to count context switches per process. Run a CPU-bound and an I/O-bound program together and compare their counts.

- 3 Measure context switch cost on your own machine: ping-pong a byte through a pipe between two processes, and then between two threads. Explain the difference.
- 4 Repeat the measurement with the two processes pinned to the same core and then to different cores. Explain both results in terms of cache behaviour.
- 5 Write a program that forks a child with a 100 MB heap and measures resident memory before and after. Then have the child write to every page and measure again. Explain the numbers using copy-on-write.
- 6 Implement a simple user-level thread library: a context structure, a switch routine, and a round-robin scheduler. Demonstrate that one blocking read() stalls all of your threads.
- 7 Create a zombie process deliberately and observe it with ps. Then kill the parent and watch init reap it.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 List six things the kernel must store per process, and say why the per-process kernel stack is necessary.
- 2 Why does a zombie exist, and when does an accumulation of them indicate a bug?
- 3 Why does switch save only callee-saved registers?
- 4 Why does xv6 switch via the scheduler rather than directly from process to process?
- 5 Give the quantitative reason a thread switch is cheaper than a process switch.
- 6 Why are fork and exec separate calls? Give a concrete thing the gap between them is used for.
- 7 Explain copy-on-write fork and say which earlier module's mechanism it depends on.
- 8 Which cost of a context switch is usually largest, and why is it invisible to naive measurement?